Lab 4 - VibeCoding: Prompt Log

Repo: SETAPESU26/17_wordle (Scenario 17 - Wordle) | LLM used: Claude | Language: Python (stdlib only)

This log summarises the prompts and the outcome of each. The complete exported chat is submitted separately as the full record.

0. Inspect the code and reproduce the bugs first

Prompt: Here is the assigned repo and the lab handout. Clone it, read the README and all source files, and run it. Before changing anything, reproduce the defects described in the README and tell me exactly what is wrong and why.

Findings:

- **Bug 1 (duplicate letters):** in `feedback.py` the yellow check is `ch in target`, which ignores how many copies the target has. Example: target `apple`, guess `eerie` gave `yellow yellow gray gray green`; apple has one 'e', already green at the end, so the first two must be gray.

- **Bug 2 (turns):** in `game.py` the loop is `for _ in range(6)`, so `continue` on an invalid guess still burns an attempt. Six invalid inputs ended the game.

- **Gap:** `words.py` only has 5-letter words, so 4- and 6-letter modes need new words.

Task 1 - Correct duplicate-letter feedback

Prompt: Fix evaluate() with standard Wordle rules: resolve exact matches (green) first, and let each occurrence in the target satisfy at most one occurrence in the guess. Keep the function signature. Explain the approach.

What changed: Pass 1 marks greens and counts the target letters not used by a green (collections.Counter). Pass 2 marks yellow left to right only while that count is above zero, decrementing it. Raises ValueError if lengths differ.

How I tested it: apple/eerie = gray gray gray gray green. stare/eerie = gray gray yellow gray green. crane/array = yellow green gray gray gray. grape/geese = green gray gray gray green. All checked by hand.

Commit message: Task 1: Fix duplicate-letter feedback (green first, count-limited yellows)

Task 2 - Complete guess handling

Prompt: Add clean win/loss handling, enforce the six accepted-guess limit, and make sure invalid guesses do not consume a turn. Handle quit and end-of-input too. Do not rewrite the project.

What changed: Replaced the for loop with while len(history) < 6. Only accepted guesses enter history, so history length is the turn count. Added is_valid(), a remaining-guesses prompt, win/loss messages, 'q' and EOF handling.

How I tested it: Invalid inputs (abc, empty, digits, too long) leave the counter unchanged. Win, loss (target revealed) and quit all end the game correctly.

Commit message: Task 2: Win/loss handling, 6-guess limit, invalid guesses don't use a turn

Task 3 - Variable word lengths

Prompt: Add 4-, 5- and 6-letter modes using in-memory words. The README's list only has 5-letter words, so add 4- and 6-letter ones, including some with repeated letters. Feedback must work for all lengths. Add a way to choose the length and validate the choice.

What changed: Added 10 four-letter and 10 six-letter words plus SUPPORTED_LENGTHS in words.py. main.py has a length menu with re-prompt on bad input, or 'python main.py 6'. Added test_feedback.py (unittest, stdlib).

How I tested it: Played all three modes. Unit tests for moon/noon, seed/eese, little/bubble, pepper/pppppp. Two of my first expected values were wrong (not the code); I re-derived them by hand and corrected the tests.

Commit message: Task 3: 4/5/6-letter modes, word lists, mode selection, unit tests

Task 4 - Guess history and session summary

Prompt: After each accepted guess, print a readable history (G = right spot, Y = wrong spot, - = absent). At the end give a game summary, and a session summary. Invalid attempts must not create history entries.

What changed: format_history() prints a numbered table after every accepted guess. summary() shows length, target, result and guesses used. main.py adds play-again and a session summary (games, wins, losses, quits, average guesses), all kept in memory.

How I tested it: Added test_game.py (10 tests): win on guess 1 and on the final (6th) guess, loss after 6, no 7th prompt, quit at start and mid-game, invalid input not consuming turns or adding history, all three lengths. 20/20 tests pass.

Commit message: Task 4: Guess history after each accepted guess, game + session summary, game tests

Verification summary

README required test	Covered by
Repeated-letter cases	test_feedback.py (apple/eerie, stare/eerie, moon/noon, pepper/pppppp ...)
Exact matches / absent letters	test_feedback.py (all green, all gray)
Invalid lengths and input	test_game.py (abc, empty, digits, too long) - no turn used, no history entry
All three word lengths	test_game.py + manual runs of modes 4, 5, 6
Win on final allowed guess	test_game.py
Losing	test_game.py (6 wrong guesses, target revealed)
Quitting	test_game.py (immediate and mid-game)

Constraints respected: standard library only, all state in memory (no CSV/JSON/SQLite), original file structure kept, no unrelated rewrite. Run tests with: `python -m unittest`